

# PRODUCT REQUIREMENTS DOCUMENT (PRD)

---

## PCA-Based Network Anomaly Detection System

---

### 1. Product Overview

#### 1.1 Problem Statement

Traditional intrusion detection systems rely heavily on **labeled attack data**, which creates a critical limitation:

- They fail when encountering **previously unseen attacks**
- They assume:

[  
 $P_{\{\text{train}\}}(X) \approx P_{\{\text{test}\}}(X)$   
]

In real-world networks, this assumption **does not hold**.

---

#### 1.2 Core Idea

This project builds an **unsupervised anomaly detection system** that:

- Learns **normal (benign) network behavior**
- Detects anomalies via **deviation from learned structure**
- Uses **PCA reconstruction error** as the anomaly signal

#### Anomaly Score:

[  
 $|x - \hat{x}|^2$   
]

---

#### 1.3 Key Research Question

Can a model trained only on benign traffic detect unseen attacks while remaining robust to distribution shift?

---

### 2. Product Goals

#### 2.1 Primary Goals

- Detect **previously unseen attacks**
  - Maintain **low false positive rate**
  - Remain robust under:
-

- temporal changes
  - environment changes
  - dataset shifts
- 

## 2.2 Secondary Goals

- Compare PCA with:
    - Isolation Forest
    - One-Class SVM
    - Autoencoder
  - Identify **failure modes of PCA**
- 

## 2.3 Non-Goals

- Not building a real-time production IDS (yet)
  - Not optimizing only for accuracy
  - Not assuming PCA is the best model
- 

# 3. System Architecture

## 3.1 High-Level Pipeline

The system processes network traffic through a structured sequence of transformations, converting raw data into anomaly decisions.

### Pipeline Flow

#### 1. **Raw Traffic**

Collection of network packets or flow-level data from the source.

#### 2. **Feature Extraction**

Convert raw traffic into structured numerical features (e.g., flow duration, packet counts, bytes/sec).

#### 3. **Preprocessing**

- Handle missing and infinite values
- Filter numerical features
- Standardize feature scales

#### 4. **PCA Model (Trained on Benign Data)**

Learn the low-dimensional representation of normal network behavior.

#### 5. **Reconstruction**

Project data into PCA space and reconstruct it back to the original feature space.

#### 6. **Reconstruction Error**

Compute the difference between original and reconstructed data:

[  
 $\text{Error}(x) = |x - \hat{x}|^2$   
 ]

## 7. Thresholding

Compare reconstruction error against a predefined threshold.

## 8. Anomaly Decision

- If error  $\leq$  threshold → **Normal**
- If error  $>$  threshold → **Anomaly**

## Compact Representation

### 3.2 Core Principle

Learn normal → detect deviation  
 NOT  
 Learn attacks → classify attacks

## 4. Functional Requirements

### 4.1 Data Layer

#### FR-1: Dataset Ingestion

- Load flow-based datasets (CSV format)
- Support:
  - CICIDS
  - UNSW-NB15

#### FR-2: Data Splitting

- Separate:
  - benign training data
  - benign test data
  - attack data

### 4.2 Preprocessing Layer

#### FR-3: Data Cleaning

- Remove:
  - missing values
  - infinite values

#### FR-4: Feature Filtering

- Keep only numerical features
- Remove:
  - constant features
  - redundant features

#### FR-5: Feature Scaling

$$z = \frac{x - \mu}{\sigma}$$

---

### 4.3 Feature Engineering Layer

#### FR-6: Feature Consistency

- Ensure compatibility across datasets
- Prevent data leakage

#### FR-7: Feature Selection Strategy

- Correlation filtering
- Variance filtering

---

### 4.4 Model Layer (PCA)

#### FR-8: PCA Training

- Train ONLY on benign data

#### FR-9: Dimensionality Reduction

- Configurable components ( $k < n$ )

#### FR-10: Reconstruction

- Reconstruct input from reduced representation

---

### 4.5 Scoring Layer

#### FR-11: Anomaly Score

$$\text{Score}(x) = \frac{1}{n} \sum (x_i - \hat{x}_i)^2$$

---

### 4.6 Thresholding Layer

### FR-12: Threshold Strategies

- Percentile-based
- Mean + standard deviation
- Future: adaptive threshold

### FR-13: Decision Rule

[  
Score(x) > T \rightarrow Anomaly  
]

---

## 4.7 Evaluation Layer

### FR-14: Metrics

- Precision
  - Recall
  - F1 Score
  - False Positive Rate
  - AUROC
  - AUPRC
- 

## 4.8 Experimentation Layer

### FR-15: Unseen Attack Testing

- Evaluate attacks not seen during training

### FR-16: Distribution Shift Testing

- Train on dataset A
- Test on dataset B

### FR-17: Per-Attack Evaluation

- Detection performance per attack type
- 

## 4.9 Baseline Comparison Layer

### FR-18: Alternative Models

- Isolation Forest
  - One-Class SVM
  - Autoencoder
- 

## 5. Non-Functional Requirements

## 5.1 Performance

- Handle datasets with **100k+ samples**

## 5.2 Reproducibility

- Fixed random seeds
- Config-driven pipeline

## 5.3 Modularity

- Components must be:
  - independently testable
  - replaceable

## 5.4 Extensibility

- Add new models easily
- Add new datasets
- Swap threshold strategies

---

## 6. Experimental Design

---

### Experiment 1 — Basic Detection

- Train on benign
- Test on benign + attack

---

### Experiment 2 — Per-Attack Analysis

- Evaluate each attack independently

---

### Experiment 3 — Unseen Attack Detection

- Train only on benign
- Test on unseen attacks

---

### Experiment 4 — Distribution Shift

- Train: Environment A
- Test: Environment B

---

### Experiment 5 — Model Comparison

- PCA vs:

- Isolation Forest
  - One-Class SVM
  - Autoencoder
- 

## 7. Success Metrics

### Primary

- High recall on attacks
- Low false positive rate

### Secondary

- Stability under distribution shift
  - Cross-dataset consistency
- 

## 8. Risks & Assumptions

### 8.1 Assumptions

- Normal traffic lies in a **low-dimensional subspace**
  - Anomalies deviate from this structure
- 

### 8.2 Risks

#### **R1: PCA is too simple**

- May fail on subtle attacks

#### **R2: Distribution shift**

- Causes high false positives

#### **R3: Threshold instability**

- Fixed thresholds may break
- 

## 9. Development Roadmap

- Phase 1 — Data Understanding
  - Phase 2 — Preprocessing
  - Phase 3 — PCA Baseline
  - Phase 4 — Evaluation
  - Phase 5 — Unseen Attack Testing
  - Phase 6 — Distribution Shift
  - Phase 7 — Model Comparison
  - Phase 8 — Failure Analysis & Improvement
-

---

## 10. Future Enhancements

- Adaptive thresholding
- Hybrid models (PCA + Autoencoder)
- Concept drift detection
- Online learning

---

## 11. Philosophy